

Affordmed Interview Preparation Guide

Technical Interview Support Document

Congratulations!

Congratulations on clearing the online assessment and being shortlisted for the technical interview.

The interview will primarily evaluate your:

- Technical fundamentals
- Problem-solving ability
- Coding skills
- Software engineering knowledge
- Communication skills
- Ability to explain your implementation and design decisions

Remember:

Interviewers are interested in your approach and reasoning as much as your final solution.

Interview Strategy

During the interview:

- Understand the problem completely before answering.
- Explain your approach before writing code.
- Think aloud while solving problems.
- Write clean and modular code.
- Discuss Time and Space Complexity.
- Consider edge cases.
- If you don't know something, be honest and explain your thought process.

1. JavaScript Fundamentals & ES6+ Features

Core JavaScript

- Variables (var, let, const)
- Scope (Global, Function, Block)
- Hoisting
- Closures
- Functions
- Arrow Functions
- Objects
- Arrays
- Event Loop
- Call Stack



- Callback Queue
- Promises
- Async/Await

ES6+ Features

- let vs const vs var
- Arrow Functions
- Template Literals
- Destructuring (Array & Object)
- Spread Operator (...)
- Rest Operator (...)
- Default Parameters
- Enhanced Object Literals
- Optional Chaining (?.)
- Nullish Coalescing (??)
- Ternary Operator
- Modules (import / export)

Array Methods

- forEach()
- map()
- filter()
- find()
- findIndex()
- reduce()
- some()
- every()
- includes()
- sort()
- slice()
- splice()
- concat()
- flat()
- flatMap()

Object Methods

- Object.keys()
- Object.values()
- Object.entries()



Data Structures

- Set
- Map

OOP Concepts

- Classes
- Objects
- Constructors
- Inheritance
- Encapsulation
- Polymorphism (Basics)

2. Backend Development

Revise:

- Express.js
- REST APIs
- HTTP Methods
- HTTP Status Codes
- Middleware
- Routing

Understand:

- req.params
- req.query
- req.body
- req.headers

3. API Integration

Be comfortable with:

- Axios
- External API Consumption
- API Aggregation
- Async/Await
- Promise.all()
- Promise.allSettled()
- Sequential API Calls
- Parallel API Calls
- Timeout Handling
- Error Handling



4. Data Structures & Algorithms

Arrays

- Traversal
- Searching
- Sorting
- Filtering

Strings

- Prefix Matching
- Pattern Searching

Hash-Based Structures

- Set
- Map

Common DSA Patterns

- Frequency Count
- Prefix Search
- Aggregation
- Statistics (Min, Max, Average)
- Searching
- Sorting
- Duplicate Removal

5. React

Revise:

- Components
- JSX
- Props
- State
- useState()
- useEffect()
- Event Handling
- Conditional Rendering
- List Rendering
- API Integration
- Component Communication

6. Full Stack Concepts

Understand:

- Frontend ↔ Backend Communication
- API Design
- Component Architecture
- Folder Structure
- Service Layer
- CORS
- Error Handling

7. Git & GitHub

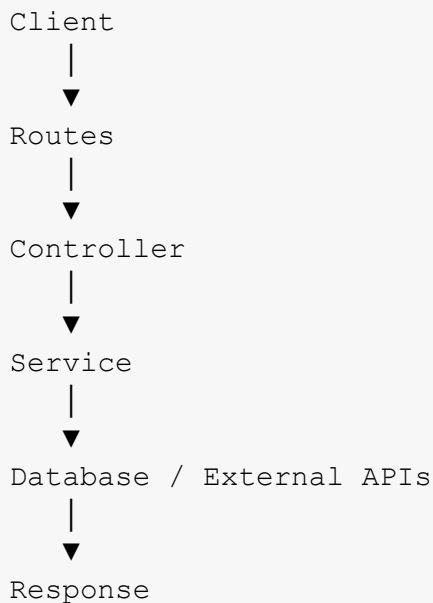
- Git Basics
- Repository Creation
- Commit & Push
- .gitignore
- Branches (Basics)

8. Software Engineering & Design Principles

The interviewer may not ask you to explain SOLID principles directly, but they are likely to evaluate whether you naturally apply good software engineering practices while designing and explaining your solution.

MVC Architecture

Understand the request flow:



Be able to explain the responsibility of each layer.



Separation of Concerns (SoC)

Each module should have a single responsibility.

Example:

- Routes → Define API endpoints
- Controllers → Handle requests and responses
- Services → Business logic
- Models → Data representation
- Components → UI Rendering

Single Responsibility Principle (SRP)

Every class, function, or component should have only one reason to change.

Example:

Instead of writing API calls directly inside a React component,

Create:

- ProductService.js
- EventService.js

Similarly,

Instead of writing business logic inside Express routes,

Move it to Service classes/functions.

DRY (Don't Repeat Yourself)

Avoid duplicate code.

Instead of:

`axios.get(...)`

`axios.get(...)`

`axios.get(...)`

Create a reusable helper function.

Example:

`fetchData(url)`

KISS (Keep It Simple)

Prefer simple and readable solutions over unnecessarily complex implementations.

Readable code is easier to maintain and debug.

R



Reusability

Design reusable modules whenever possible.

Examples:

Backend

- Utility Functions
- Service Layer
- Middleware

Frontend

- ProductCard
- FilterBar
- Pagination
- SearchBar

Component-Based Architecture (React)

Split the UI into reusable components.

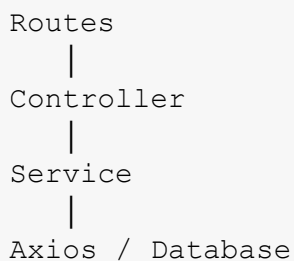
Example:



Service Layer Pattern

Avoid placing API calls inside React components or route handlers.

Instead:



Benefits:

- Reusable code
- Easier testing
- Better maintainability
- Cleaner architecture



Clean Folder Structure

```
Backend
routes/
controllers/
services/
middlewares/
utils/
models/
Frontend
components/
pages/
services/
hooks/
assets/
```

Code Quality

Follow good coding practices:

- Meaningful variable names
- Small functions
- Proper indentation
- Modular code
- Exception handling
- Input validation
- Consistent naming conventions

Engineering Mindset

When discussing your solution, always explain:

- Why you selected this approach.
- Why you used a particular data structure.
- Why you separated the code into multiple files.
- How your design improves readability and maintainability.
- How your solution can be extended if new requirements are introduced.

Remember:

Interviewers are not only evaluating whether your code works—they are also evaluating whether another developer can easily understand, maintain, and extend your code.

Be Ready to Explain Your Code

You should be able to explain:

- Your overall approach
- Why you selected a particular data structure
- Why you chose a specific algorithm



- Why you used Promise.all()
- When Promise.allSettled() is preferred
- Error handling strategy
- API Design
- Time Complexity
- Space Complexity

Remember:

Writing code is only half the interview. Explaining your implementation is equally important.

Frequently Asked Interview Questions

JavaScript

- Difference between var, let and const
- Difference between == and ===
- null vs undefined
- Arrow Function vs Normal Function
- map() vs forEach()
- filter() vs find()
- Spread vs Rest Operator
- Shallow Copy vs Deep Copy
- Promise vs Async/Await
- Promise.all() vs Promise.allSettled()
- Call, Apply and Bind
- Event Loop
- Closures
- Hoisting

Backend

- What is REST?
- Difference between GET and POST
- PUT vs PATCH
- HTTP Status Codes
- What is Middleware?
- req.params vs req.query vs req.body
- What is CORS?
- Why Express.js?



API Integration

- Why Axios?
- How do you consume external APIs?
- How do you call multiple APIs simultaneously?
- How do you handle API failures?
- How do you implement timeout handling?
- How would you optimize multiple API calls?

React

- Explain useState()
- Explain useEffect()
- Props vs State
- Component Lifecycle
- Controlled Components
- How do you call APIs in React?

DSA

- Arrays
- Strings
- Set
- Map
- Sorting
- Searching
- Prefix Matching
- Frequency Count
- Time Complexity
- Space Complexity

Project Discussion

Be prepared to answer:

- Explain your project architecture.
- Explain your folder structure.
- Explain your API design.
- Why did you choose this approach?
- How did you handle errors?
- What improvements would you make?
- What challenges did you face?



- How did you debug the application?

Coding Best Practices

While coding:

- Clarify the requirements.
- Explain your approach.
- Think about edge cases.
- Write modular code.
- Use meaningful variable names.
- Handle exceptions.
- Test your solution before submitting.

Communication Tips

- Speak confidently.
- Explain your thought process.
- Don't rush.
- Ask clarifying questions if needed.
- Stay calm if you get stuck.
- Demonstrate logical thinking.

Before the Interview

- Revise the assessment solutions.
- Practice explaining your code aloud.
- Revise JavaScript ES6+ concepts.
- Revise Express.js and React fundamentals.
- Practice common DSA patterns.
- Ensure GitHub repositories are updated.
- Keep your development environment ready.
- Ensure a stable internet connection (if online).

Prediction

Topic	Probability
JavaScript	★★★★★★
Express	★★★★★★
React	★★★★☆
REST APIs	★★★★★★
Promise	★★★★★★
API Aggregation	★★★★★★
DSA	★★★★☆
OOP	★★★☆☆
Folder Structure	★★★★☆
Basic System Design	★★★★☆
SOLID (SRP only)	★★★☆☆
Complete SOLID	★☆☆☆☆
LLD	★★★☆☆
HLD	★☆☆☆☆

Final Advice

The interview is not only about writing the correct code.

Interviewers evaluate:

- Technical knowledge
- Problem-solving ability
- Communication skills
- Software engineering practices
- Confidence in explaining your implementation

Stay calm, think logically, communicate clearly, and justify your technical decisions.

Good luck, and wish you success in your Affordmed technical interview!